

Asher Erickson
Colin Truran
Elise Longenecker
Yvan Longin

Applied Analytics Project - Week 6 Assignment

Model Approach:

For Week 6, our group selected Logistic Regression as our primary modeling approach. This decision aligns directly with our prediction objective of predicting NCAA tournament game outcomes as the model estimates the probability that a binary outcome equals one of two classes. For our project, the target variable represents whether Team 1 wins the matchup (1) or loses (0).

Logistic regression makes sense for this task because:

- The outcome is binary (win or loss).
- The model estimates probabilities between 0 and 1.
- It is interpretable and allows us to understand how features affect the odds of winning.

Model Complexity:

Logistic regression is a relatively simple model. It looks at all the features, multiplies them by weights, adds them together, and then turns that number into a probability between 0 and 1.

Compared to more complex models like neural networks or tree-based models, logistic regression is easier to understand and easier to control. That makes it a good choice for testing hyperparameters and understanding how our features affect predictions.

Although the base model is simple, we increased the depth of our analysis by testing many different hyperparameter combinations.

Hyperparameters Evaluated:

We evaluated multiple hyperparameters to determine which configuration produced the best validation performance.

The main hyperparameters tested were:

- **Regularization strength (6):** Controls how flexible the model is.
 - We tested different values to determine whether stronger or weaker regularization improved validation performance.
- **Penalty type (L1 vs L2) (2):** Determines how the model penalizes large coefficients.
 - We tested both options to see which one helps the model make better predictions without overfitting.
- **Class weights (4):** Adjusts how much importance is given to correctly predicting wins versus losses.
 - We tested this to see if giving extra importance to wins improves performance.
- **Probability threshold (1):** Determines the cutoff for classifying a prediction as a win.
 - We kept the threshold at 0.5.
- **Tolerance and intercept settings(2 and 2):** Influence how the model converges and whether a bias term is included.
 - We tested them to see if they make any difference in performance.

In total, 192 model configurations were tested.

Model Performance Metrics:

To evaluate our Logistic Regression model, we used the following performance metrics:

- Accuracy
 - This gives a general understanding of how often the model correctly predicts the outcome of games.
- Precision
 - This is important because we want to limit false win predictions, especially since incorrect win predictions can significantly impact the outcome of brackets.
- Recall
 - This is important because we want the model to successfully capture as many true wins as possible rather than missing them.
- Confusion Matrix
 - This helps us understand the types of mistakes the model is making.

Model Variations:

For this week, we evaluated 192 different Logistic Regression configurations using a structured hyperparameter grid. Each configuration was trained on seasons 2003–2021 and validated using the 2022 season. For every model, we calculated accuracy, precision, and recall on the training and validation datasets.

	configId	model	set	accuracy	precision	recall
0	0	{'cValue': 0.1, 'classWeight': None, 'fitInter...	train	0.674699	0.675302	0.997449
1	0	{'cValue': 0.1, 'classWeight': None, 'fitInter...	val	0.634921	0.634921	1.000000
2	1	{'cValue': 0.1, 'classWeight': None, 'fitInter...	train	0.674699	0.675302	0.997449
3	1	{'cValue': 0.1, 'classWeight': None, 'fitInter...	val	0.634921	0.634921	1.000000
4	2	{'cValue': 0.1, 'classWeight': None, 'fitInter...	train	0.681583	0.687500	0.968112
...	...	...	...	...	...	...
379	189	{'cValue': 1.5, 'classWeight': {0: 1, 1: 2}, '...	val	0.603175	0.741935	0.575000
380	190	{'cValue': 1.5, 'classWeight': {0: 1, 1: 2}, '...	train	0.611876	0.781726	0.589286
381	190	{'cValue': 1.5, 'classWeight': {0: 1, 1: 2}, '...	val	0.571429	0.709677	0.550000
382	191	{'cValue': 1.5, 'classWeight': {0: 1, 1: 2}, '...	train	0.608434	0.778342	0.586735
383	191	{'cValue': 1.5, 'classWeight': {0: 1, 1: 2}, '...	val	0.571429	0.709677	0.550000

We selected the top three best-performing models by ranking all configurations based on validation accuracy and choosing the three with the highest validation performance.

Discuss and analyze how training and validation metrics change across variations:

Across the top three configurations (34, 35, and 64), training performance was slightly higher than validation performance, which is expected since the model is trained directly on the training data. Training accuracy was around 0.682, while validation accuracy was 0.651 for all three models. This small gap suggests that the models are not heavily overfitting.

Precision and recall were also very similar between training and validation. Training precision was about 0.69, while validation precision was 0.65. Training recall ranged from about 0.96 to 0.97, and validation recall was 0.975 for all three

models. This shows that the models consistently identify most true wins and perform similarly on new data.

Overall, the top three configurations produced identical validation results.

Compare performance metrics for the validation dataset across all 3 variations:

The top three models were tied on validation performance. All three top models achieved the same validation results:

- Validation Accuracy: 0.651
- Validation Precision: 0.650
- Validation Recall: 0.975

	configId	model	set	accuracy	precision	recall
68	34	{'cValue': 0.3, 'classWeight': None, 'fitInter...	train	0.681583	0.690257	0.957908
69	34	{'cValue': 0.3, 'classWeight': None, 'fitInter...	val	0.650794	0.650000	0.975000
70	35	{'cValue': 0.3, 'classWeight': None, 'fitInter...	train	0.682444	0.690542	0.959184
71	35	{'cValue': 0.3, 'classWeight': None, 'fitInter...	val	0.650794	0.650000	0.975000
128	64	{'cValue': 0.6, 'classWeight': None, 'fitInter...	train	0.682444	0.686769	0.973214
129	64	{'cValue': 0.6, 'classWeight': None, 'fitInter...	val	0.650794	0.650000	0.975000

	configId	model	set	accuracy	precision	recall
69	34	{'cValue': 0.3, 'classWeight': None, 'fitInter...	val	0.650794	0.65	0.975
71	35	{'cValue': 0.3, 'classWeight': None, 'fitInter...	val	0.650794	0.65	0.975
129	64	{'cValue': 0.6, 'classWeight': None, 'fitInter...	val	0.650794	0.65	0.975

Since these models performed identically on validation, there was no meaningful difference in their validation metrics.

Identify the best model for the week and discuss why you selected that model:

The best model selected for Week 6 was configId 34, because it ranked at the top of the validation results when sorting by validation accuracy, followed by recall, then precision. Even though the top three models had tied validation results, configId 34 was selected because it was the first-ranked configuration under our sorting rule.